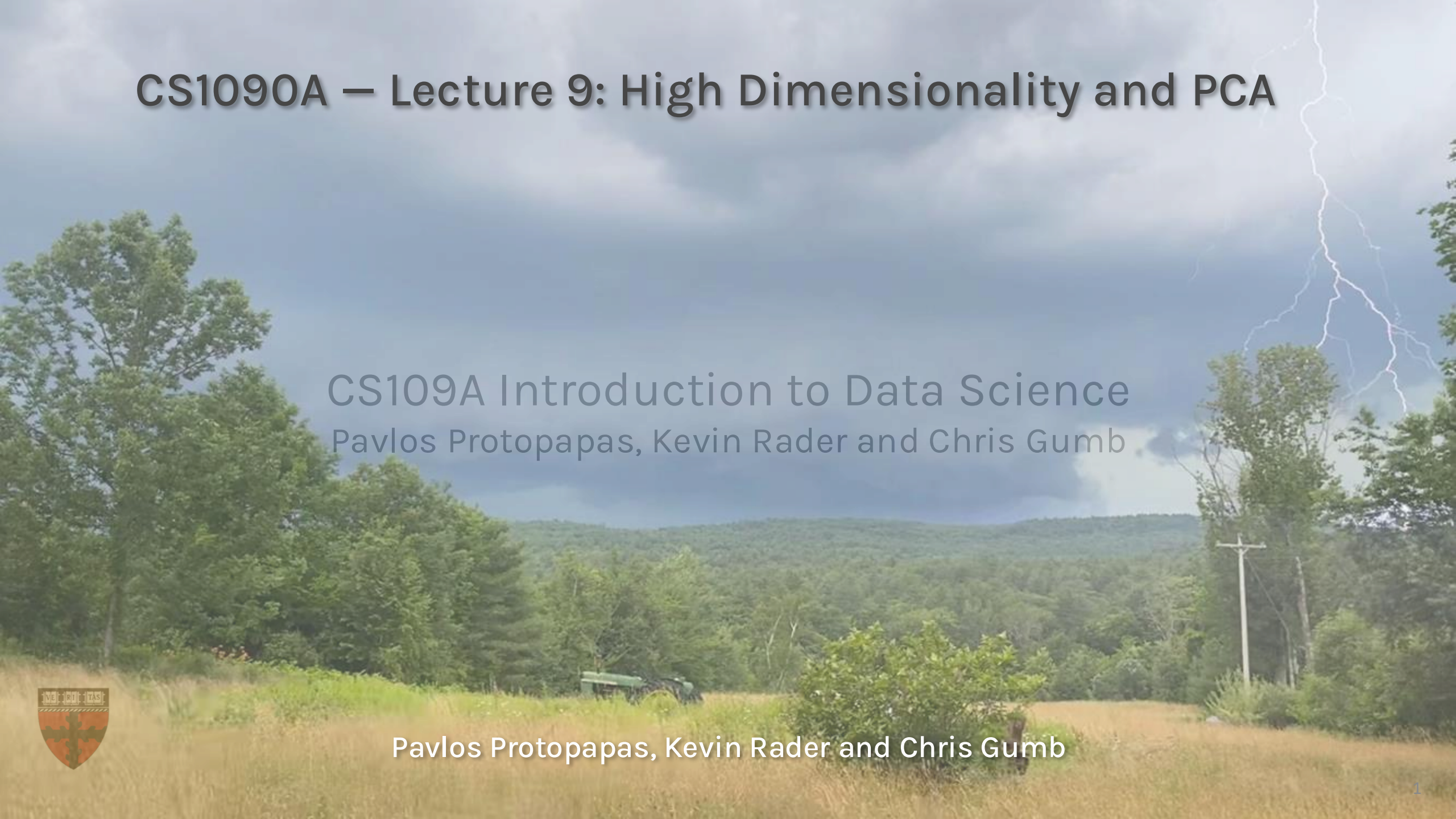
CS109OA — Lecture 9: High Dimensionality and PCA



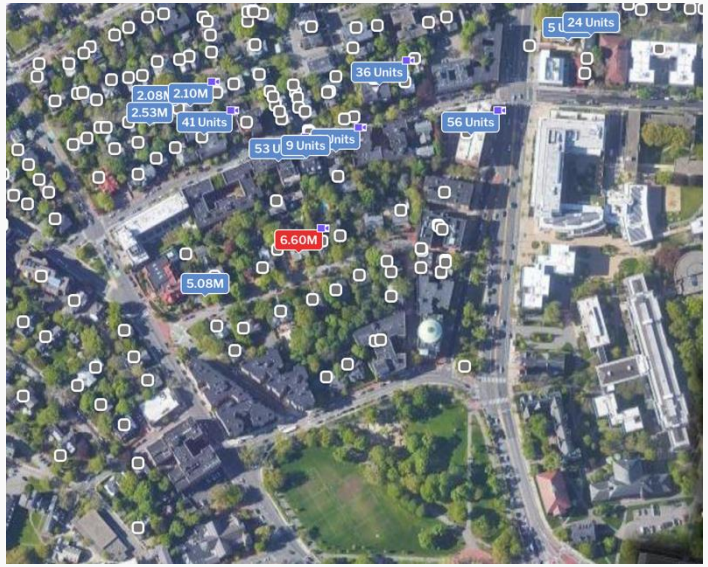
Pavlos Protopapas, Kevin Rader and Chris Gumb

Lecture Outline: High Dimensionality and PCA

- Big Data and High Dimensionality
- Principal Components Analysis (PCA)
- PCA for Visualization
- PCA for Regression (PCR)

Today's Data Science Example

Today's Data are from Redfin.com:



A place to find historical selling prices of homes in any location (and plotted on a map). Today we're looking at the Cambridge & Somerville area, residential properties, that have sold in the last 12 months.

Address	Location	Price	Beds	Baths	Sq.Ft.	$/Sq.Ft.	On Redfin
20A Lafayette S...	East Arlington	$1,425,000	4	4.5	2,503	$569	—
45 Marlboro St	Belmont	$1,590,000	5	4	4,194	$379	—
28 Betts Rd	Belmont	$1,250,000	3	3	1,913	$653	—
97 Shaw Rd	Shaw Estates	$2,186,000	4	2.5	2,844	$769	—

Ask an interesting question

Get the Data

Explore the Data

Model the Data

Communicate/Visualize the Results

*Note: Kevin did a *little* preprocessing

Today's Data Science Example

- **Response (y):**
  - Selling price (in \$1000s) for $n = 592$ homes around Cambridge and Somerville
- **Predictors (X):**
  - Condo, single family, multi-family, or townhouse
  - Number of bedrooms
  - Number of bathrooms
  - Floor space in square feet
  - Lot size in square feet
  - The year the home was built
  - Distance to Harvard Sq T-stop (in km)

```
print(homes.dtypes)
```

```
date            object
type            object
address         object
city            object
zip              int64
price            int64
beds             int64
baths          float64
neighborhood    object
sqft             int64
lotsize        float64
year           float64
hoa            float64
url             object
mls              int64
latitude       float64
longitude      float64
dist           float64
dtype: object
```

Ask an interesting question

Get the Data

Explore the Data

Model the Data

Communicate/Visualize the Results

Today's Data Science Example

Explore the Data

Notice anything concerning?
What does this indicate? What should we do?

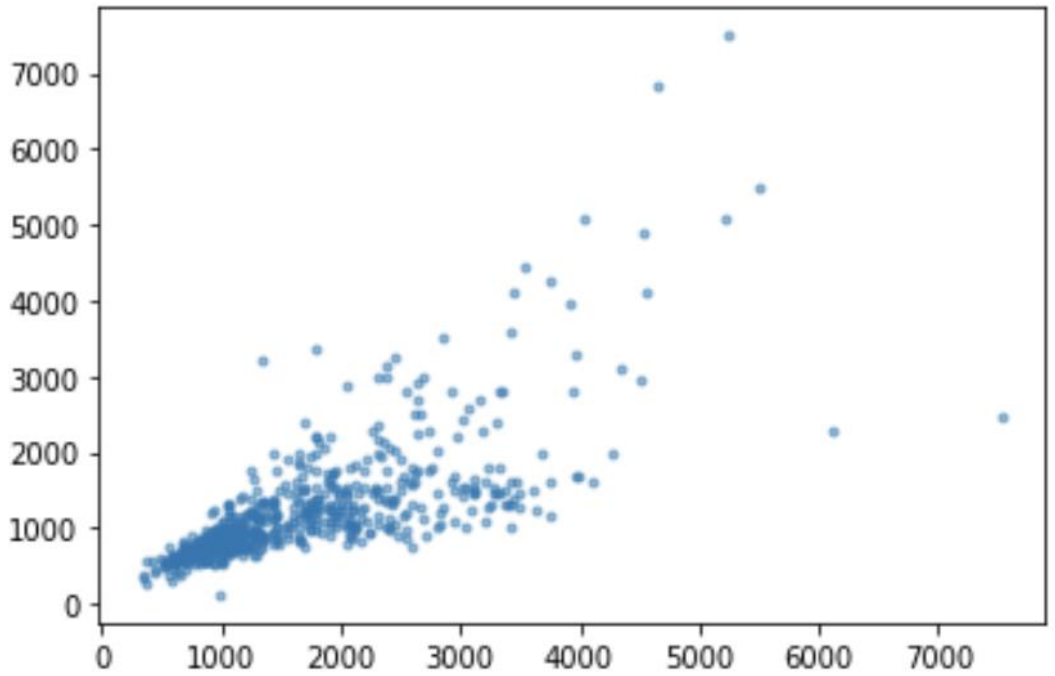
```
homes["zip"]=homes["zip"].astype(str)
homes.describe().round(5)
```

	price	beds	baths	sqft	lotsize	year	hoa	mls	latitude	longitude	dist
count	5.920000e+02	592.00000	592.00000	592.00000	183.00000	591.00000	402.00000	5.920000e+02	592.00000	592.00000	592.00000
mean	1.244197e+06	2.92568	2.05828	1690.04561	3860.93989	1926.58545	352.30100	7.308775e+07	42.38443	-71.11148	2.17682
std	7.810677e+05	1.56242	0.96753	953.69978	1858.56503	46.57422	262.84504	3.623761e+04	0.01220	0.01653	0.88593
min	1.227940e+05	0.00000	1.00000	336.00000	1007.00000	1828.00000	1.00000	7.291816e+07	42.35605	-71.16066	0.44139
25%	8.000000e+05	2.00000	1.00000	1000.00000	2557.50000	1900.00000	200.00000	7.305637e+07	42.37497	-71.12367	1.50989
50%	1.028000e+06	3.00000	2.00000	1407.00000	3580.00000	1910.00000	286.00000	7.309318e+07	42.38459	-71.10973	2.07152
75%	1.450000e+06	3.00000	2.50000	2162.00000	4562.50000	1947.50000	443.50000	7.311370e+07	42.39333	-71.10031	2.82374
max	7.500000e+06	9.00000	7.00000	7530.00000	10454.00000	2024.00000	2984.00000	7.316201e+07	42.41408	-71.07172	4.60435

Today's Data Science Example

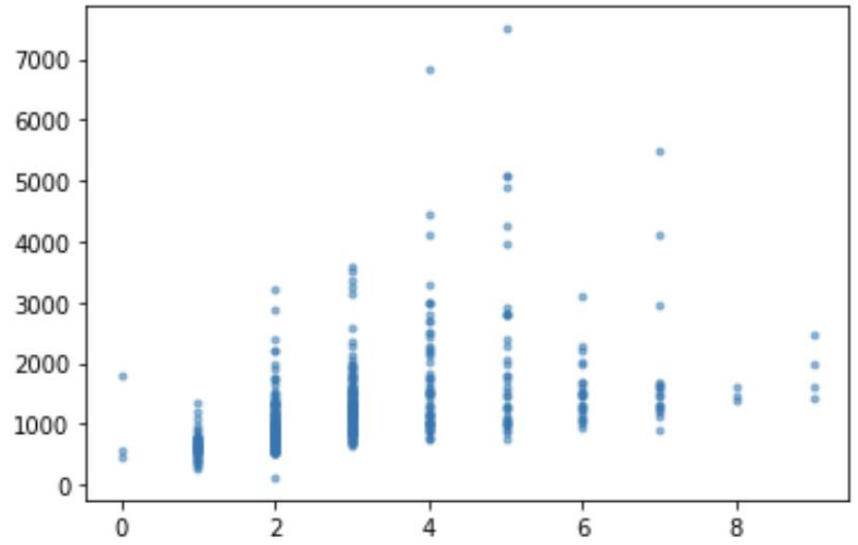
Explore the Data

```
plt.scatter(x = homes['sqft'], y = homes['price'],
            alpha = 0.5, marker = '.');
```

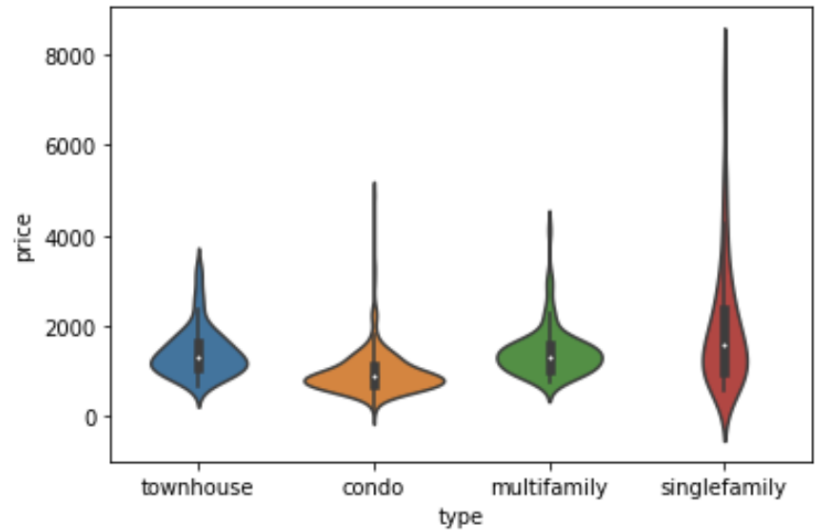


What do you notice? Any causes for concern?

```
plt.scatter(x = homes['beds'], y = homes['price'],
            alpha = 0.5, marker = '.');
```




```
sns.violinplot(x = homes['type'], y = homes['price']);
```



Today's Data Science Example

Model the Data

```
type_dummies = pd.get_dummies(homes['type'], drop_first=True)
homes=pd.concat([homes,type_dummies],axis=1)
```

```
X_full = homes[['multifamily', 'singlefamily', 'townhouse',
                'sqft',  'dist', 'beds', 'baths', 'year']]
regress_full = sk.linear_model.LinearRegression().fit(X = X_full, y = homes['price'])

pd.DataFrame({'coef': np.append(regress_full.intercept_,regress_full.coef_)},
             index = np.append("intercept",X_full.columns))
```

	coef
intercept	-1949.067039
multifamily	-452.235208
singlefamily	335.761220
townhouse	-76.437171
sqft	0.641074
dist	-173.542970
beds	-89.934486
baths	198.464604
year	1.229999

What are the coefficient interpretations?

Notice anything interesting? What does this indicate?

```
regress_sqft = sk.linear_model.LinearRegression().fit(X = homes[['sqft']], y = homes['price'])
print("Intercept =",regress_sqft.intercept_.round(2),", Slope =",regress_sqft.coef_[0].round(4))

regress_beds = sk.linear_model.LinearRegression().fit(X = homes[['beds']], y = homes['price'])
print("Intercept =",regress_beds.intercept_.round(2),", Slope =",regress_beds.coef_[0].round(4))
```

```
Intercept = 247.44 , Slope = 0.5898
Intercept = 570.39 , Slope = 230.3084
```

Today's Data Science Example

Model the Data

The library **`statsmodels`** makes our life WAY simpler (my R is showing):

```
import statsmodels.api as sm
import statsmodels.formula.api as smf
```

```
fullmodel_sm = smf.ols(formula = "price ~ sqft + type + dist + beds + baths + year",
                       data = homes).fit()
fullmodel_sm.params
#fullmodel_sm.summary()
```

```
Intercept                -1927.782817
type[T.multifamily]       -455.751842
type[T.singlefamily]       332.773270
type[T.townhouse]          -78.756789
sqft                         0.641839
dist                      -173.274319
beds                       -90.246936
baths                      199.724130
year                         1.218183
dtype: float64
```

Lecture Outline: Probability

- Today's Example
- **Review**
- Probability and Random Variables
  - Normal Distribution
  - Binomial Distribution
- Likelihood
- Statistical Inference

What is ‘Big Data’?

In the world of Data Science, the term *Big Data* gets thrown around a lot. What does *Big Data* mean?

A rectangular data set has two dimensions: number of observations (n) and the number of predictors (p). Both can play a part in defining a problem as a *Big Data* problem.

What are some issues when:

- n is big (and p is small to moderate)?
- p is big (and n is small to moderate)?
- n and p are both big?

When *n* is big

A very large sample size can pose computational a challenge.

- Algorithms can take forever to finish. Estimating the coefficients of a regression model, especially one that does not have closed form (like LASSO), can take a while.
- Model selection and hyperparameters tuning, especially when using cross-validation, exacerbates the problem.

What can we do to fix this computational issue?

- Perform 'preliminary' steps (model selection, tuning, etc.) on a subset of the training data set. 10% or less can be justified.

Note: statistical inference is not important when *n* is VERY big.

Keep in mind, big *n* doesn't solve everything

The era of Big Data (aka, large *n*) can help us answer lots of interesting scientific and application-based questions, but it does not fix everything.

Remember the old adage: "**garbage in; garbage out**". If the data are not representative of the population, then modeling results can be terrible. Random sampling ensures representative data.

Xiao-Li Meng does a wonderful job describing the subtleties involved (WARNING: it's a little technical, but digestible): https://www.youtube.com/watch?v=8YLdIDOMEZs

When p is big

When the number of predictors is large (many interactions, polynomial terms, etc.), lots of issues can occur.

What are some issues that may arise?

- Matrices may not be invertible (issue in OLS).
- Multicollinearity is likely to be present
- Models are susceptible to overfitting

This situation is called *High Dimensionality* and needs to be accounted for when performing data analysis and modeling.

When Does High Dimensionality Occur?

The problem of high dimensionality can occur when the number of parameters exceeds or is close to the number of observations. This can happen when we consider lots of interaction terms, like in our previous example. But this can also happen when the number of main effects is high.

For example:

- When we are performing polynomial regression with a high degree and a large number of predictors.
- When the predictors are genomic markers (and possible interactions) in a computational biology problem.
- When the predictors are the counts of all English words appearing in a text.

How Does sklearn handle unidentifiability?

In a parametric approach: if we have an over-specified model ($p > n$), the parameters are unidentifiable: we only need $n - 1$ predictors to perfectly predict every observation ($n - 1$ because of the intercept).

So, what happens to the 'extra' parameter estimates (the extra $\beta's$)?

The remaining $p - (n - 1)$ predictors' coefficients can be estimated to be *anything!* Thus, there are an infinite number of sets of estimates that will give us identical predictions. There is not one unique set of $\beta's$!

Perfect Multicollinearity

The $p > n$ situation leads to perfect collinearity of the predictor set. But this can also occur with redundant predictors.

Let's see what sklearn does in this situation:

```
regress_sqft = sk.linear_model.LinearRegression().fit(X = homes[['sqft']], y = homes['price'])
print("Intercept =",regress_sqft.intercept_.round(2),", Slope =",regress_sqft.coef_[0].round(4))

regress_sqft2 = sk.linear_model.LinearRegression().fit(X = homes[['sqft','sqft']], y = homes['price'])
print("Intercept =",regress_sqft2.intercept_.round(2),", Slopes =",regress_sqft2.coef_.round(4))
```

```
Intercept = 247438.24 , Slope = 589.7823
Intercept = 247438.24 , Slopes = [294.8911 294.8911]
```

Lecture Outline: High Dimensionality and PCA

- Big Data and High Dimensionality
- **Principal Components Analysis (PCA)**
- PCA for Visualization
- PCA for Regression (PCR)

In high dimensional space, data are sparse

Consider the example where we add many interaction and/or polynomial terms to our design matrix.

As the number of dimensions increase the feature space is soon dominated by empty space.

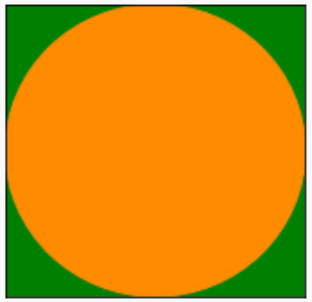
We say that the resulting feature space is ‘sparse.’

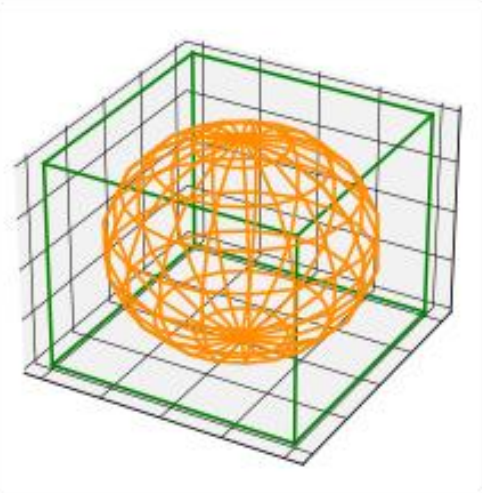
One consequence is that the number of data points required for your machine learning algorithms to remain effective increases exponentially.

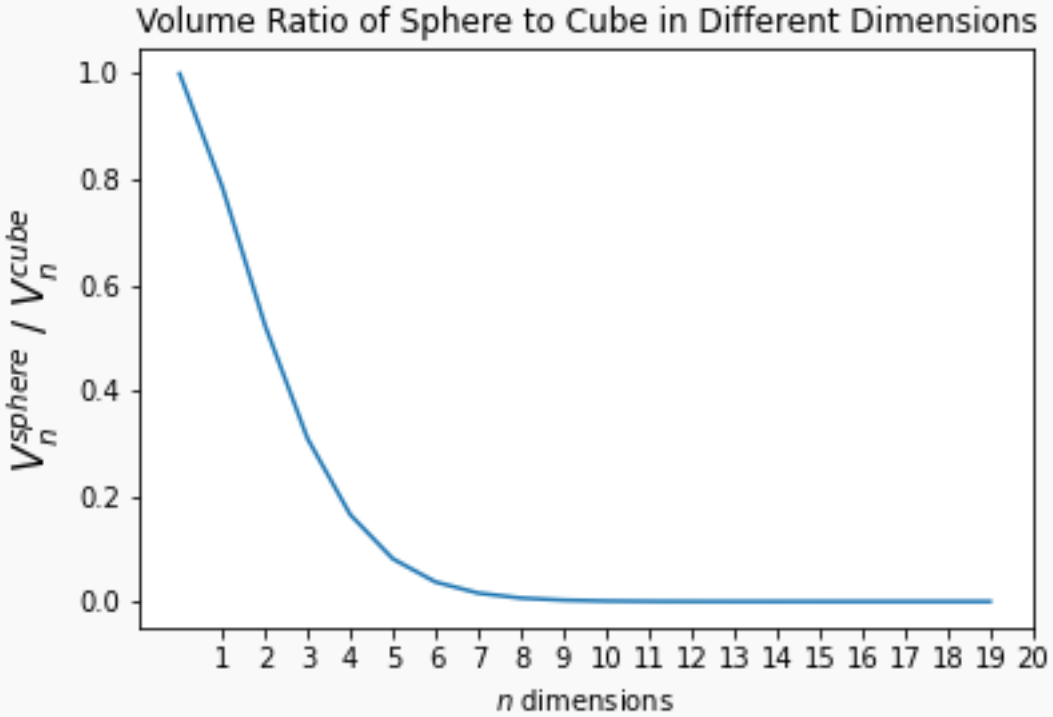
Don’t trust your spatial intuitions in high dimensions

Strange things happen in high dimensions, as illustrated by the example of the volume ration of an *n*-sphere to an *n*-cube rapidly *vanishing to zero* as *n* increases. This is disconcerting, but what implications does it have for doing data science?

$$V_n^{sphere} = \frac{\pi^{\frac{n}{2}}}{\Gamma\left(\frac{n}{2}+1\right)} \qquad V_n^{cube} = 2^n$$







In high dimensional space, there are no ‘neighbors’

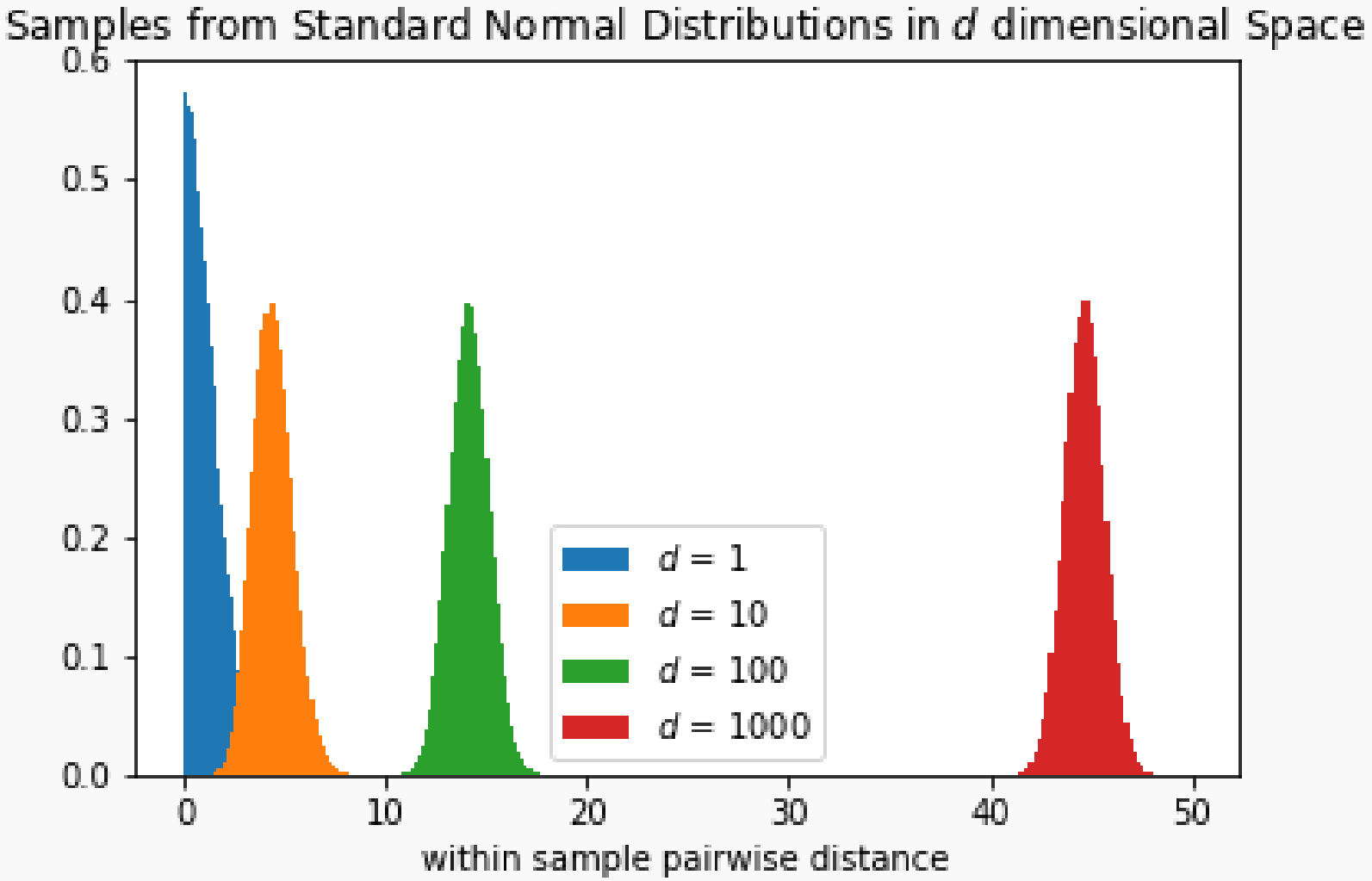
Many algorithms rely on the notion that nearby points in the feature space are ‘similar.’ And this similarity is used to inform predictions.

k-NN is one such algorithm where this idea of nearness = similarity is quite explicit.

It’s also true that, as the dimensionality increases, the calculation of the pairwise distances required by *k*-NN quickly become intractable as there are more dimensions to consider in each distance calculation. But this is the least of your worries!

In high dimensional space, all points are far away from one another. The notion of ‘neighbors’ begins to break down.

It’s lonely in high dimensional space

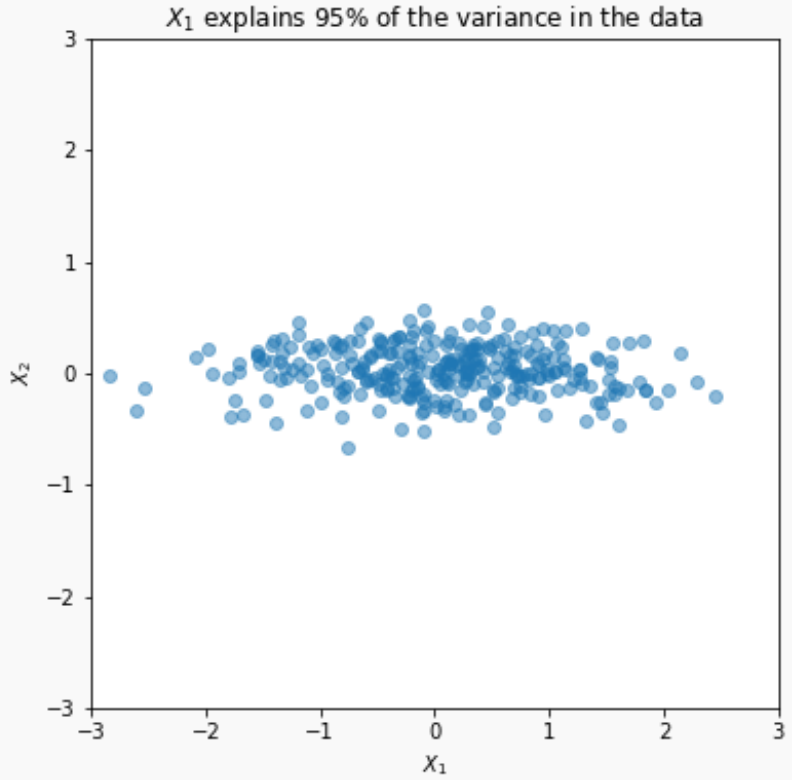


High Dimensionality and Overfitting

- More predictors means more degrees of freedom.
- The more degrees of freedom, the easier it is for our model to fit to noise in the training data, leading to poor generalization. This is overfitting.
- If some predictors are highly correlated, then there is redundancy in the dataset.
- Eliminating redundant predictors and those providing little to no new information will help prevent overfitting while retaining as much of the original information as possible.

Which predictors should be removed or retained?

One strategy is to remove those predictors which explain the least amount of variance in the data, or, equivalently, retain those *X*s that explain the most variance of the predictor set.

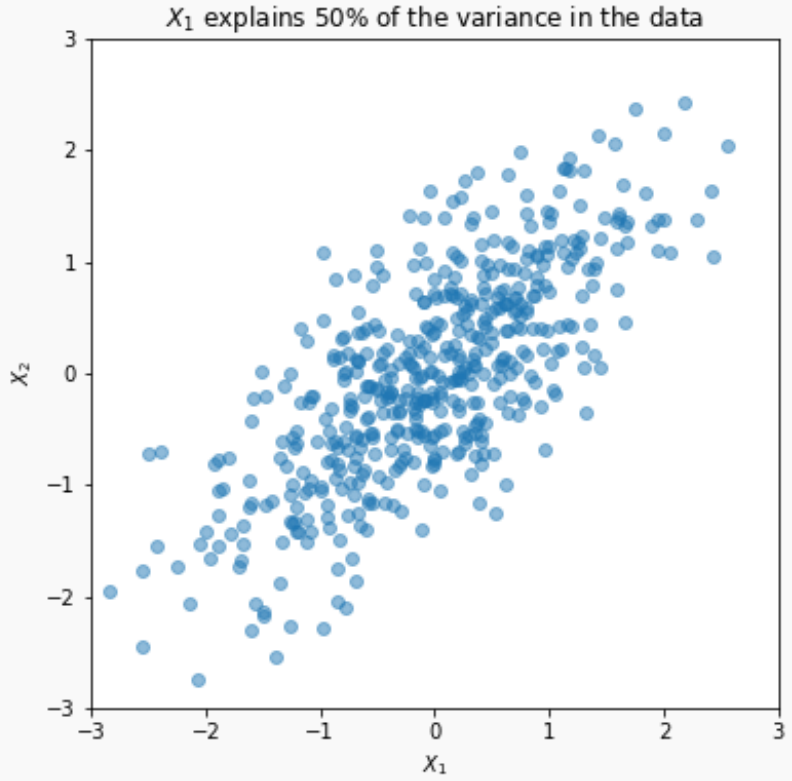


In this example, almost all the variance of the predictors is in X_1.

Keeping only this predictor would retain most of the information in the predictor set while cutting the dimensionality in half.

An example with moderate correlation

If the variance explained by our predictors is very similar, then this metric will not help us decide which predictor(s) to keep.

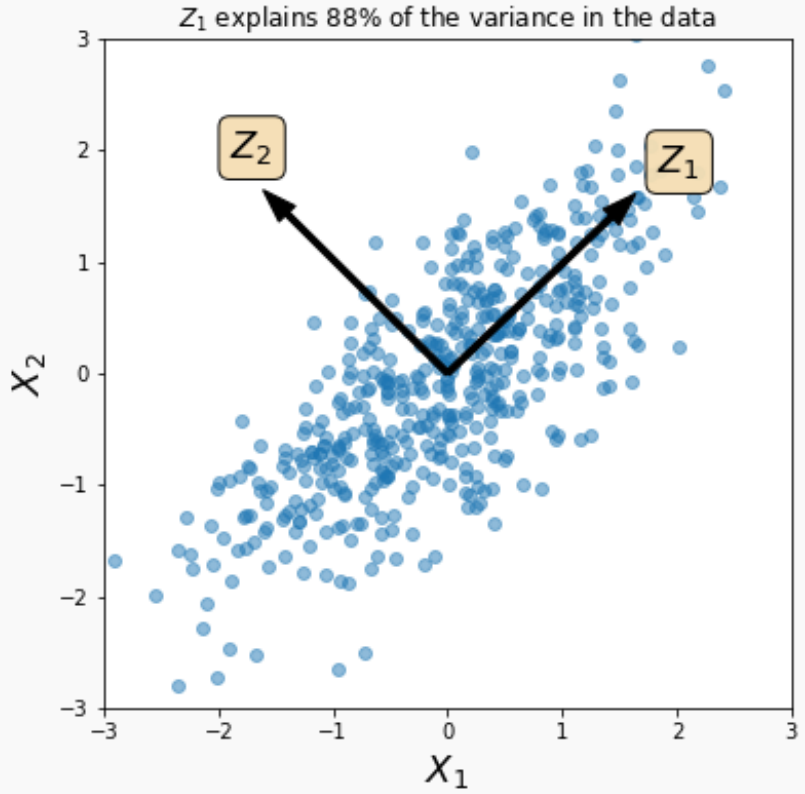


Here the variances of X_1 and X_2 are the same!

But we can clearly see there are *some directions in the predictor space* that have more spread than others. They just don't lie along the axes (i.e., the basis vectors)

Directions of maximum variance

The vector Z_1 represents the direction of maximum variance in the predictor space. Z_2 is orthogonal and captures the remaining unexplained variance.

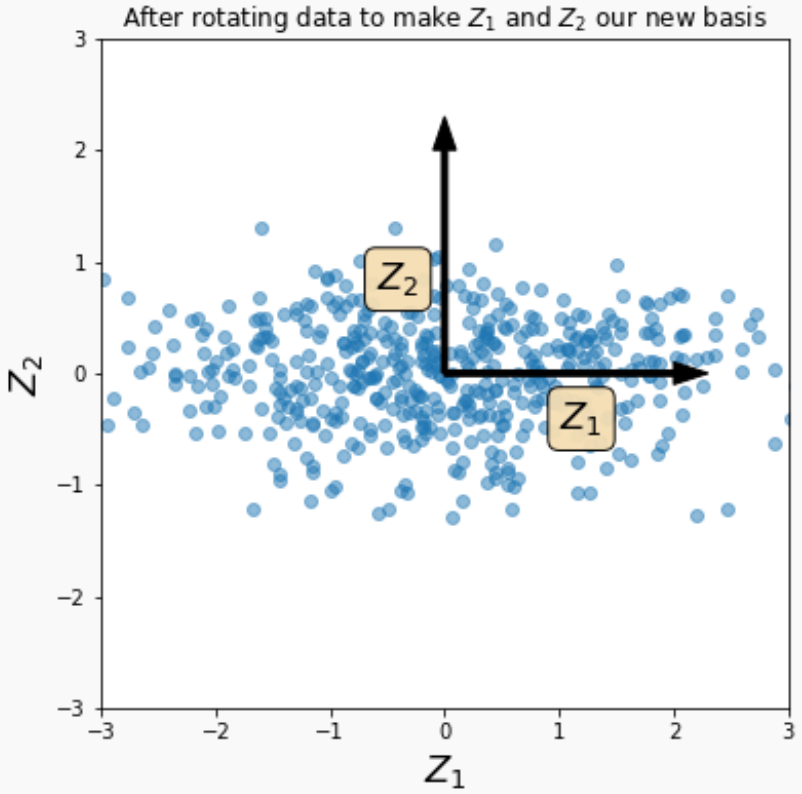


If our data were represented in terms of Z_1 and Z_2 then we could decide to keep only the Z_1 component.

This would again cut the dimensionality in half while retaining the most information.

Rotation for a change of basis

We can rotate our data so that the orthogonal Z_1 and Z_2 vectors now lie along the axes. This is equivalent to a change of basis from X_1 and X_2 to Z_1 and Z_2



We can think of Z_1 and Z_2 as *a new set of predictors*. The decision of which predictor to keep to retain the most variance is once again clear.

Great! But how did we discover the optimal vectors Z_1 and Z_2!?

A Framework for Linear Dimensionality Reduction

Our strategy will be to create a new, smaller set of predictors by taking linear combinations of the original predictors.

For $m < p$ we choose $Z_1, Z_2, \ldots, Z_m$, where each Z_i is a linear combination of the original p predictors:

$$Z_i = \sum_{j=1}^{p} \phi_{ji} X_j$$

for fixed constants Φ_{ji}. Then we can build a linear regression regression model using the new predictors

$$Y = \beta_0 + \beta_1 Z_1 + \cdots + \beta_m Z_m + \varepsilon$$

Notice that this model has a smaller number ($m+1 < p+1$) of parameters.

A Framework for Linear Dimensionality Reduction (cont.)

Our method of dimensionality reduction includes 2 steps:

1. Determine an optimal set of new predictors $z_1, \ldots, z_m$ for $m < p$.
2. Express each observation in the data in terms of these new predictors. The transformed data will have m columns rather than p.

Thereafter, we can fit a model using the new predictors.

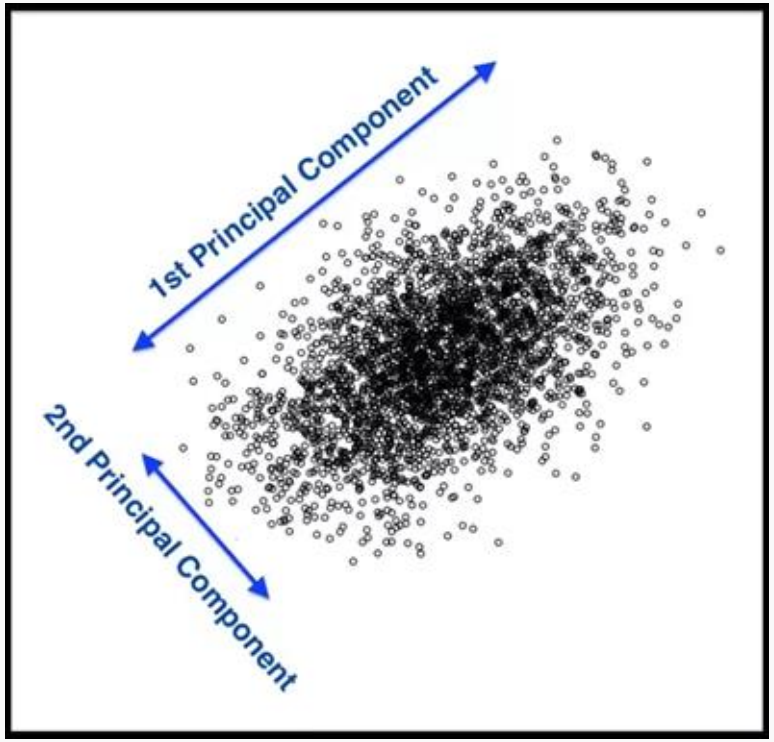
The method for deciding what is meant by an "optimal" set of new predictors can differ according to application. We will continue to explore a way to create new predictors that captures the **most variation** in the observed predictor set.

Big data = Bad Time



Principal Components Analysis (PCA)

Principal Components Analysis (PCA) is a method to identify a new set of predictors, as linear combinations of the original ones, that captures the maximum amount of variance in the observed data (predictors).



PCA (cont.)

PCA produces a list of p **principal components** $Z_1,\ldots, Z_p$ such that:

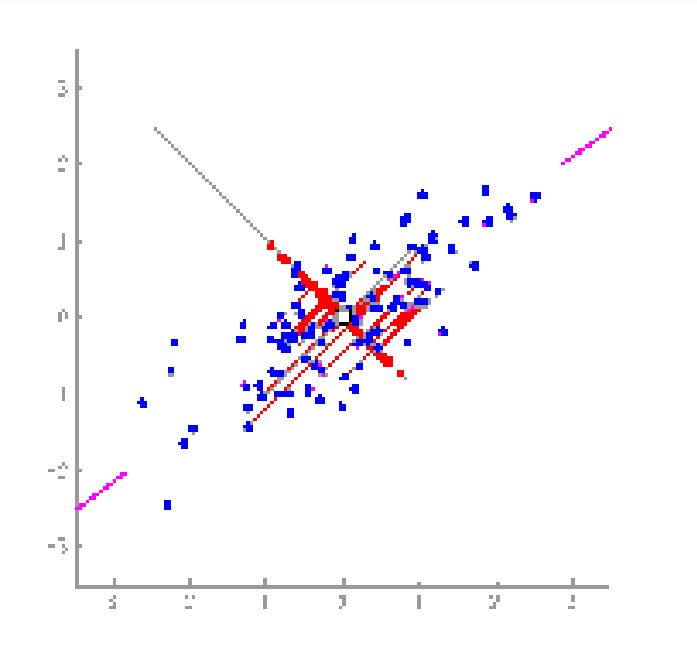
I. Each Z_i is a linear combination of the original predictors
II. The Z_i's are orthonormal (pairwise orthogonal with unit length)
III. The Z_i's are ordered based on the amount of variance explained

That is, the observed data shows more variance in the direction of Z_1 than in the direction of Z_2.

To perform dimensionality reduction, we select the top m principal components and express our data in terms of these predictors.

The Intuition Behind PCA (cont.)

Transforming our observed data means **projecting** our dataset onto the space defined by the top *m* PCA components. These components become our new predictors.



The animation shows projections onto several candidate vectors in red.

You can see the points retain the most spread when projected onto what turns out to be the first PCA component.

The Math behind PCA (for those whose are curious)

Let $\boldsymbol{X}$ be the n x p matrix with columns X_1,..., X_p (our original predictors), each standardized to have mean zero and variance one, and without the intercept)

Let let $\boldsymbol{W}$ be the p x p matrix whose columns are the eigenvectors of the covariance matrix, $\boldsymbol{X}^T\boldsymbol{X}$

Let $\boldsymbol{Z}$ be the n x p matrix with columns Z_1,..., Z_p (the full set of principal components):

$$\mathbf{Z}_{n\times p} = \mathbf{X}_{n\times p}\mathbf{W}_{p\times p}$$

Implementation of PCA using linear algebra (optional)

To implement PCA yourself you can perform the following steps:

I. [ideally] standardize your predictors (so they each have mean = 0, sd = 1).

II. Calculate the eigenvectors of $\boldsymbol{X^T X}$ and and arrange them as columns in order of descending eigenvalues in a new matrix, $\boldsymbol{W}$.

III. Use matrix multiplication to project $\boldsymbol{X}$ onto the eigenvectors to create the new feature matrix, $\boldsymbol{Z = XW}$

Note: this is not efficient from a computational perspective. This can be sped up using Cholesky decomposition.

PCA is easy to perform in Python using sklearn's decomposition.PCA class.

PCA example in sklearn

```
X = homes[['sqft','beds','baths','lotsize','dist']]

pca = PCA().fit(X)

pcaX = pca.transform(X)
W = pca.components_.T


print("First PCA Component vector (w1):",W[0,:].round(7))
print("Second PCA Component vector (w2):",W[1,:].round(7))

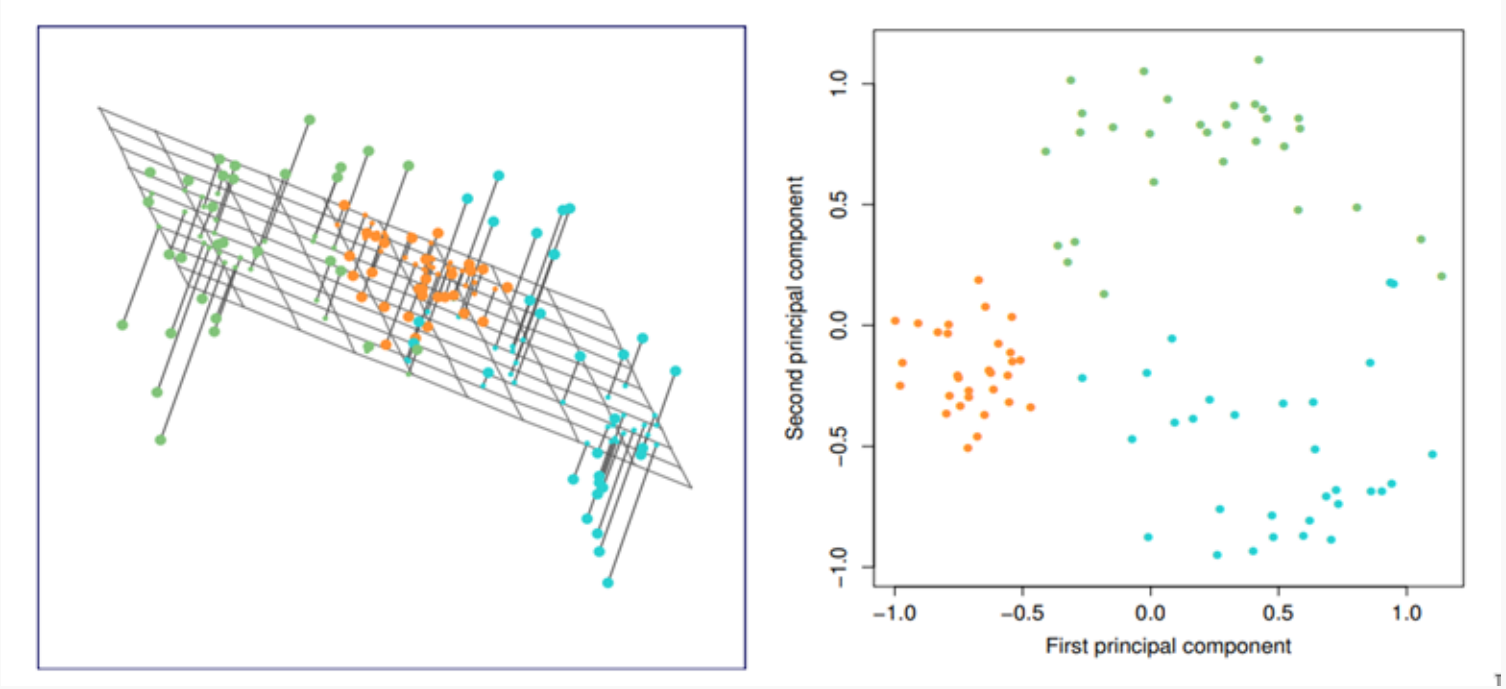
print("Variance explained by each component:",pca.explained_variance_ratio_.round(7))
```

```
First PCA Component vector (w1): [ 3.237999e-01  9.461243e-01 -7.237000e-04  9.225000e-04  1.009800e-03]
Second PCA Component vector (w2): [ 4.902000e-04  1.144900e-03  6.274349e-01 -7.753376e-01 -7.194070e-02]
Variance explained by each component: [9.097255e-01 9.027410e-02 2.000000e-07 1.000000e-07 1.000000e-07]
```

An Alternative Interpretation of PCA

We've seen an interpretation of PCA as finding the directions in the predictor space along which the data varies the most.

An alternative interpretation is that PCA finds a low-dimensional linear surface which is *closest* to the data points.



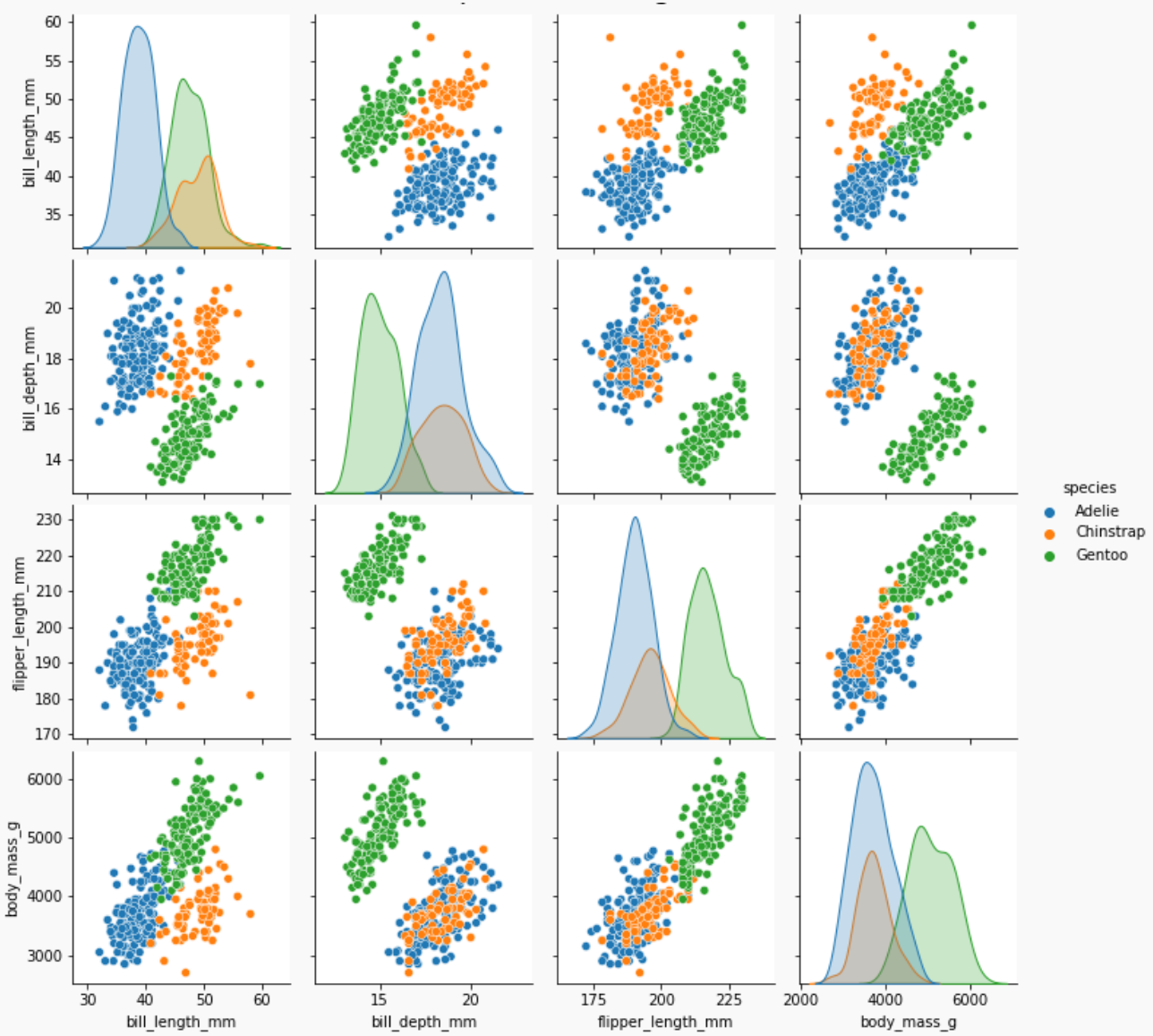
Lecture Outline: High Dimensionality and PCA

- Big Data and High Dimensionality
- Principal Components Analysis (PCA)
- **PCA for Visualization**
- PCA for Regression (PCR)

Visualizing high dimensional data

- Visualization is often the quickest ways to gain insight into the relationships and patterns within a dataset.
- But how can we possibly visualize data beyond 2 or 3 dimensions?
- One option is to compromise and look only at small ‘slices’ of the predictor space at time.
- For example, we can create a series of pair-wise 2-D scatter plots between all the predictors.

3 Penguin Species: Pair-plot of features

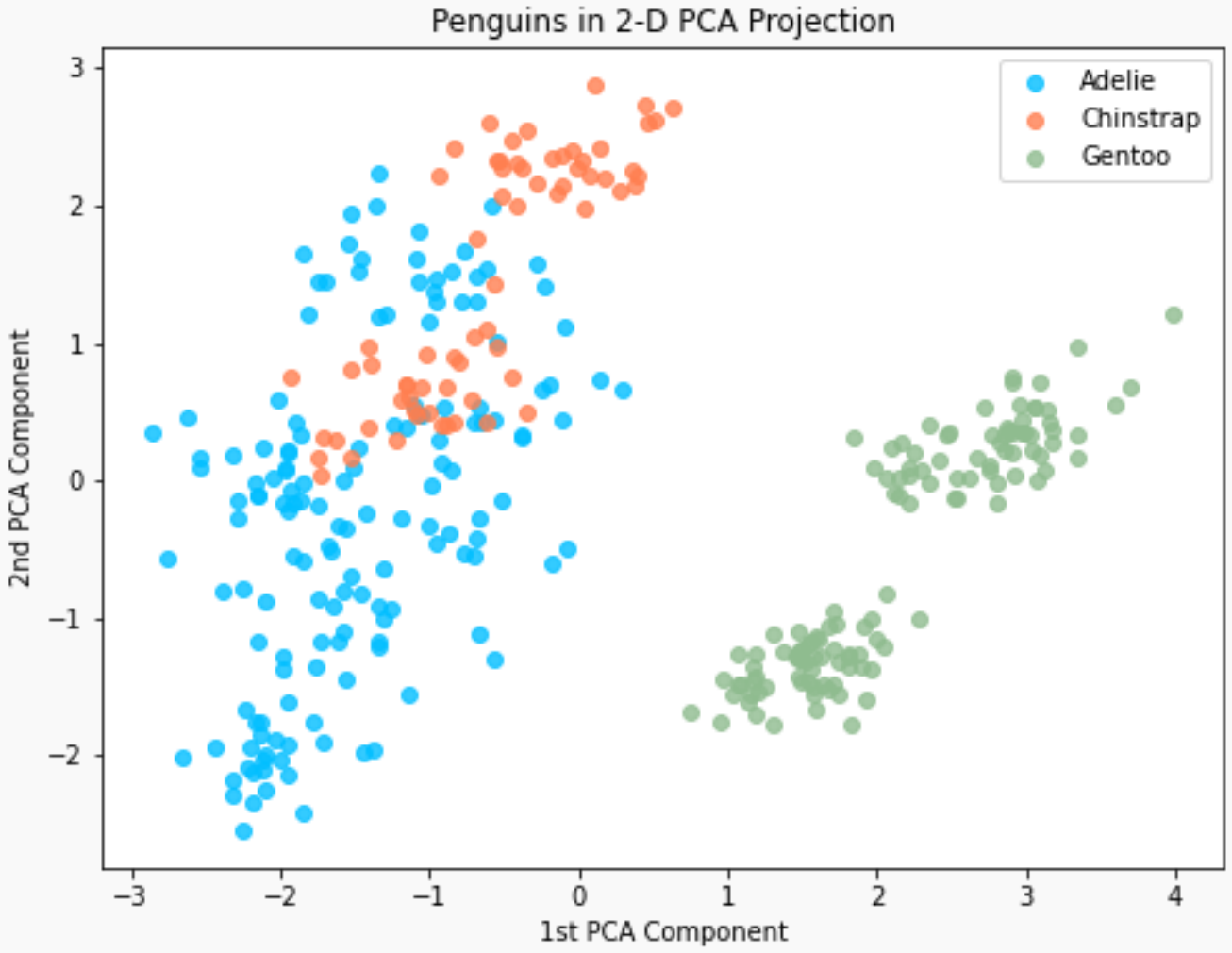


We have 4 measurements (4 *X*s) from 3 species of penguins: body mass, flipper length, bill length, and bill depth.

We can see some separation of the species in a few of the scatter plots. But what about relationships involving more than just 2 predictors?

This approach also becomes unwieldy with more than just a few predictors.

3 Penguin Species: Visualized with PCA



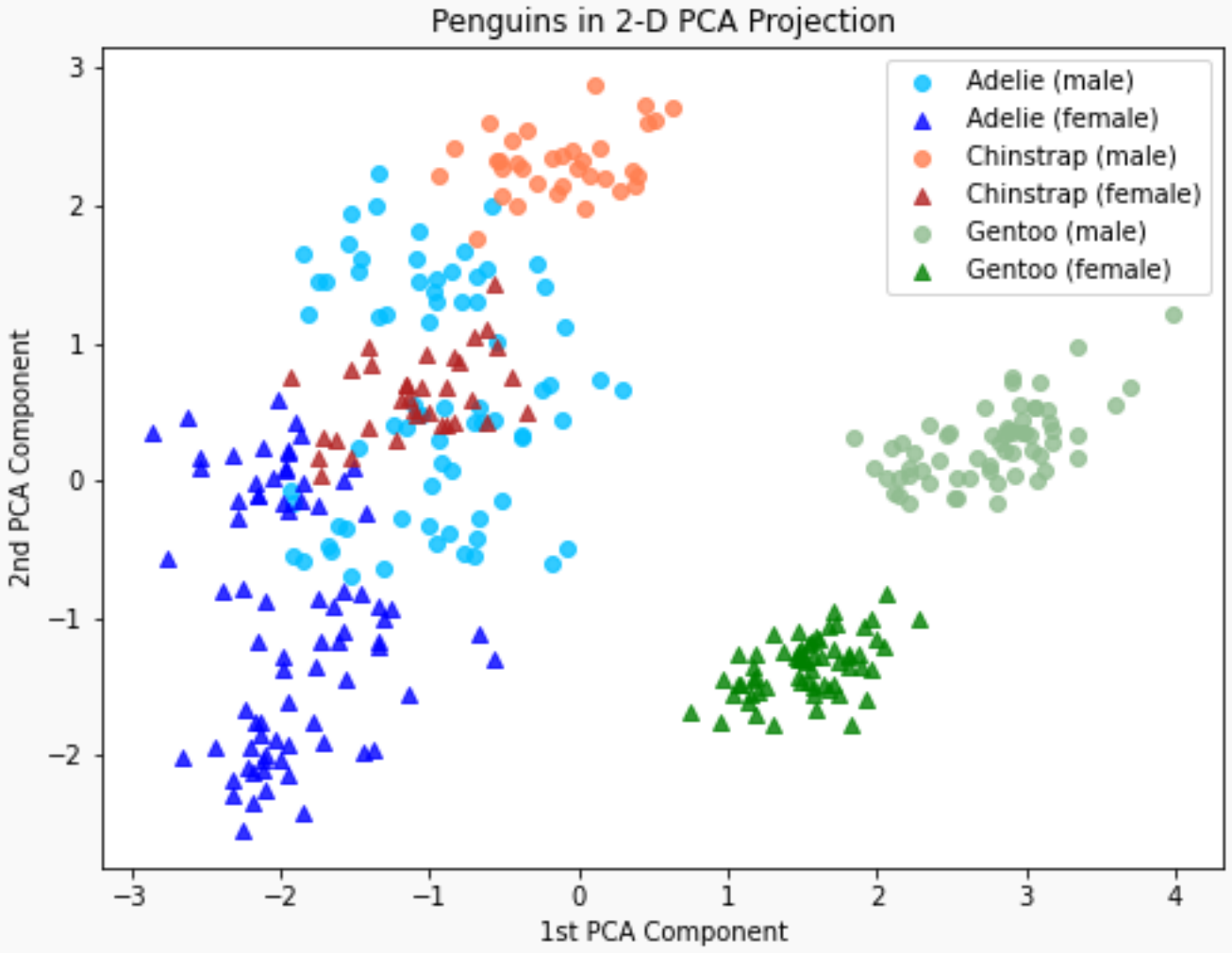
Here is the data projected onto the first 2 principle components.

Perhaps you're not convinced this is an improvement?

It might not look too different from the previous plots.

But notice that there are also some distinct clusters within species…

3 Penguin Species: Visualized with PCA



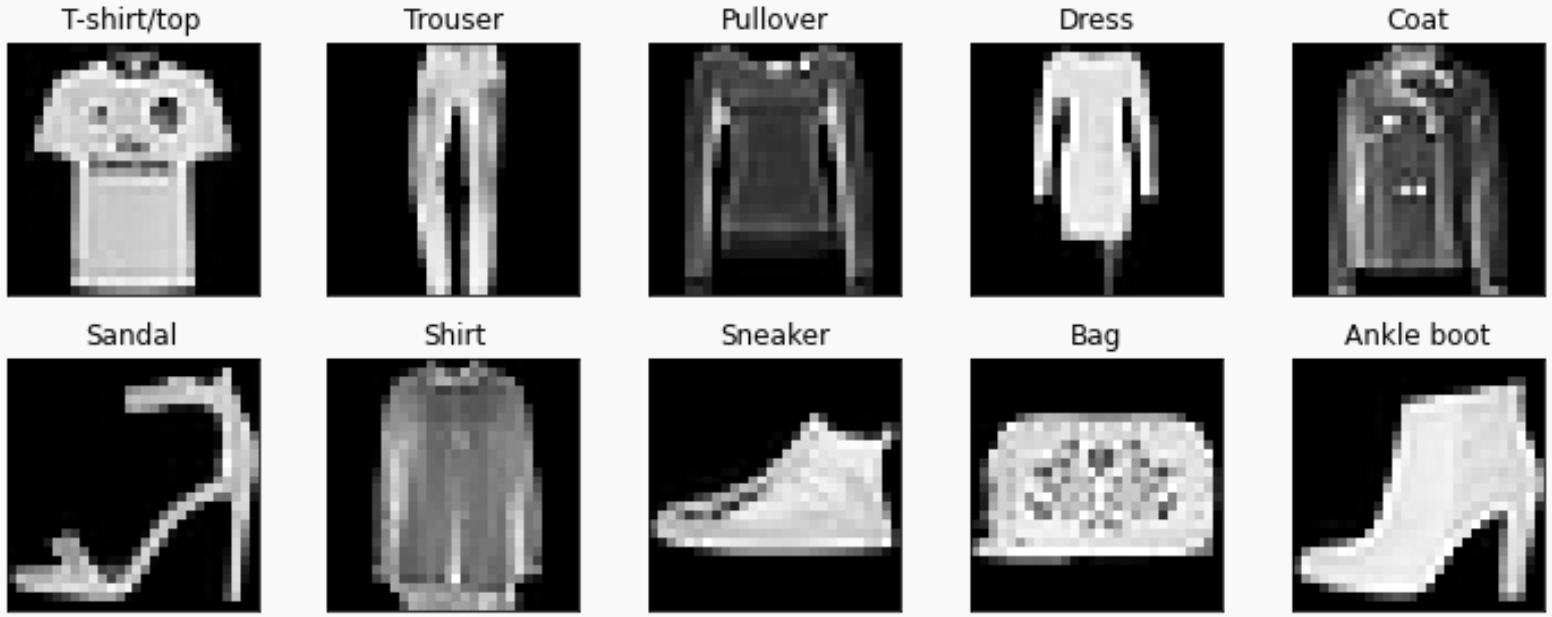
It turns out that these within species clusters correspond to male and female penguins.

Now let's try another example with a much higher dimensional dataset…

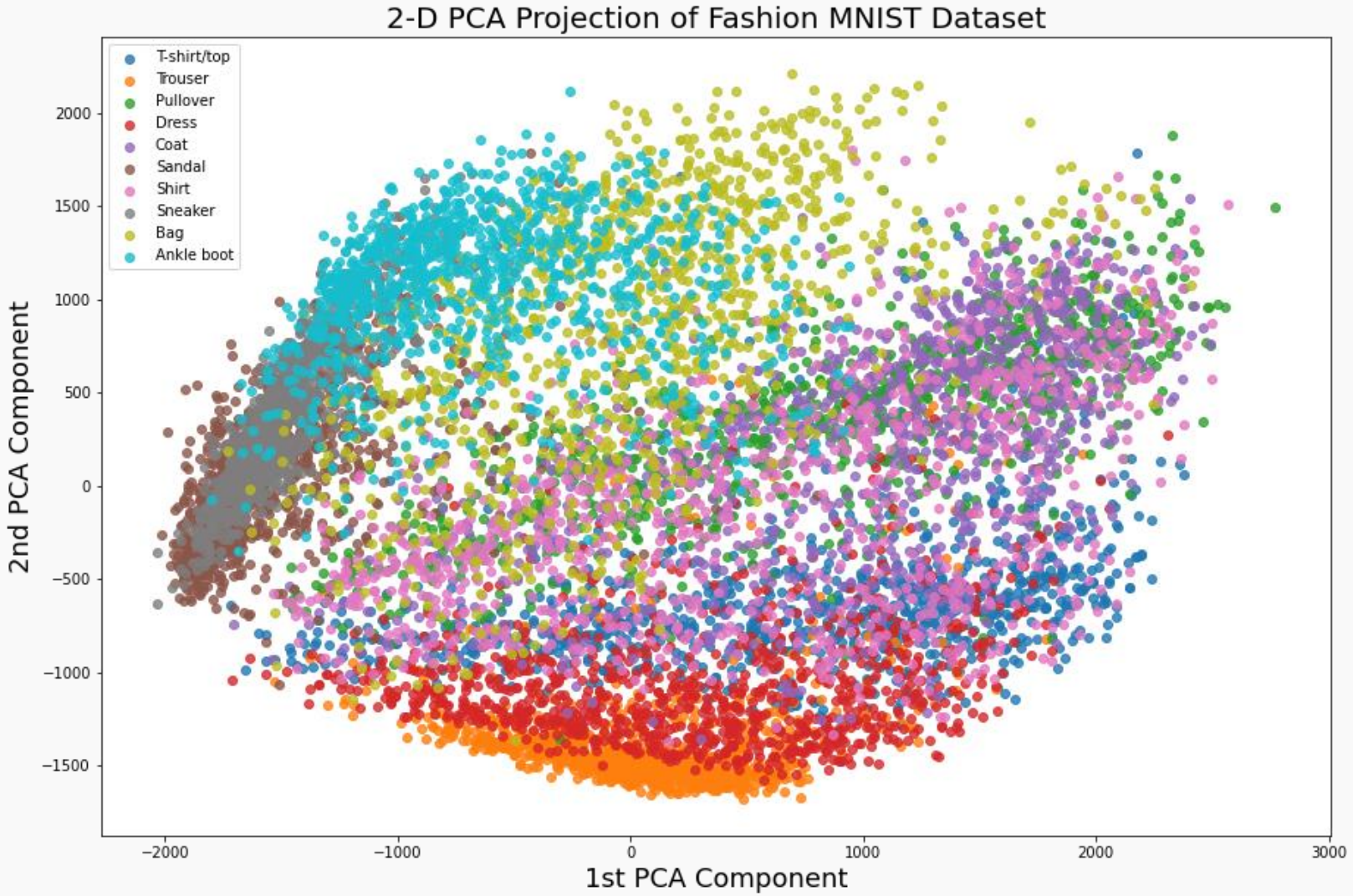
Images as high dimensional data

The **Fashion MNIST** data set consists of thousands of 28x28 grayscale images of articles of clothing from 10 different categories. Each pixel is essentially a feature. 28 x 28 = 784 features!

And obviously plotting pair-wise scatter plots of pixel values wouldn't be insightful.



PCA for visualizing image data

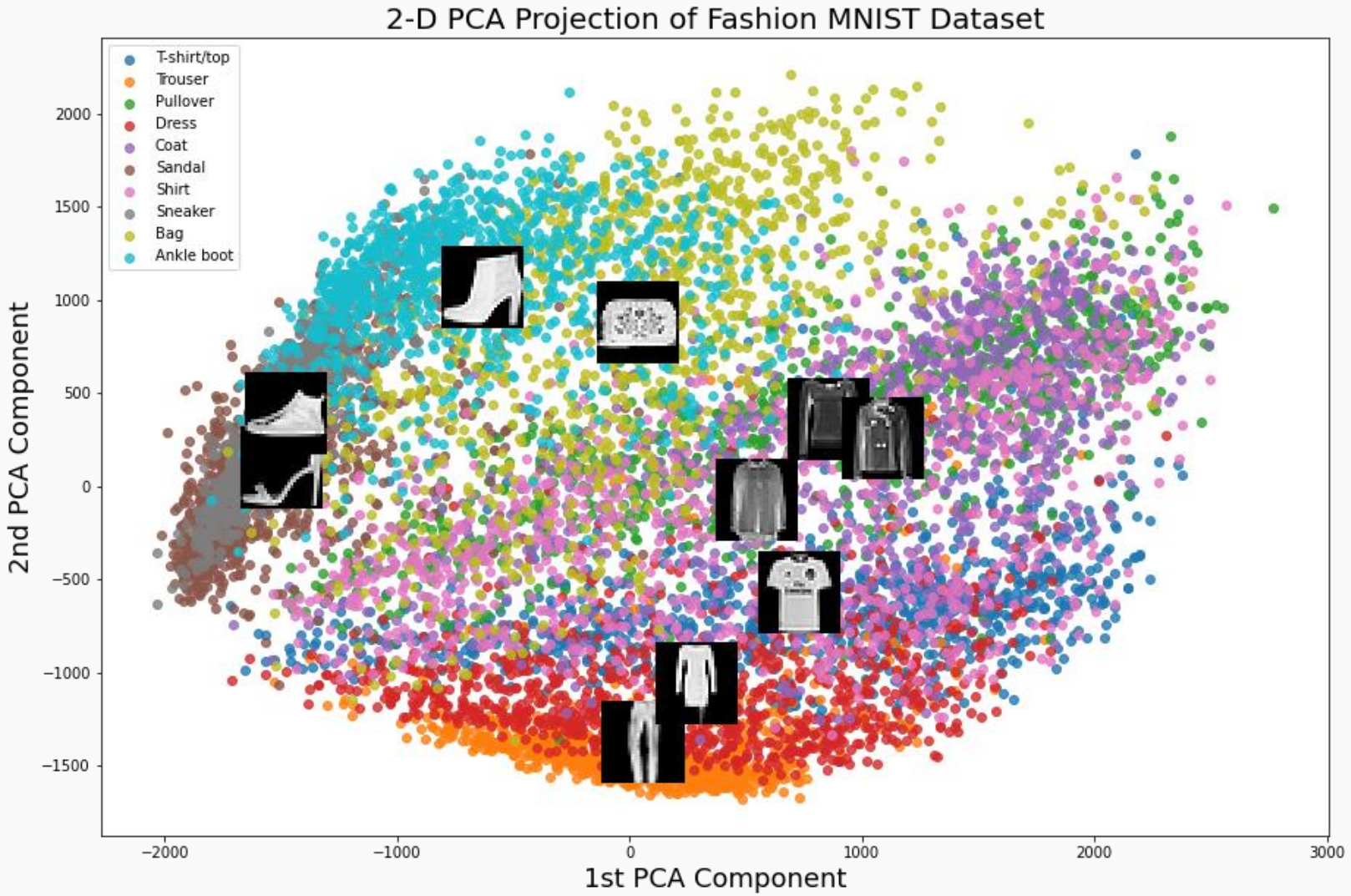


Projecting onto the top 2 principal components allows us to visualize the dataset.

While there exists considerable overlap, we see that most categories are grouped together.

And remember, PCA never actually saw the category labels!

PCA for visualizing image data



Displaying examples at the center of their respective cluster we can appreciate that similar categories are close to one another in the projected space.

All this was done by finding linear combinations of pixels that explained the most variance in the data!

Lecture Outline: High Dimensionality and PCA

- Big Data and High Dimensionality
- Principal Components Analysis (PCA)
- PCA for Visualization
- **PCA for Regression (PCR)**

PCA for Regression (PCR)

PCA is easy in Python, so how can we use it for regression modeling in a real-life problem?

If we use all p of the new Z_i, then we have *not* reduced the dimensionality. To do so, we must select only the first m PCA variables, $Z_1,\ldots,Z_m$, to use as predictors for some $m < p$.

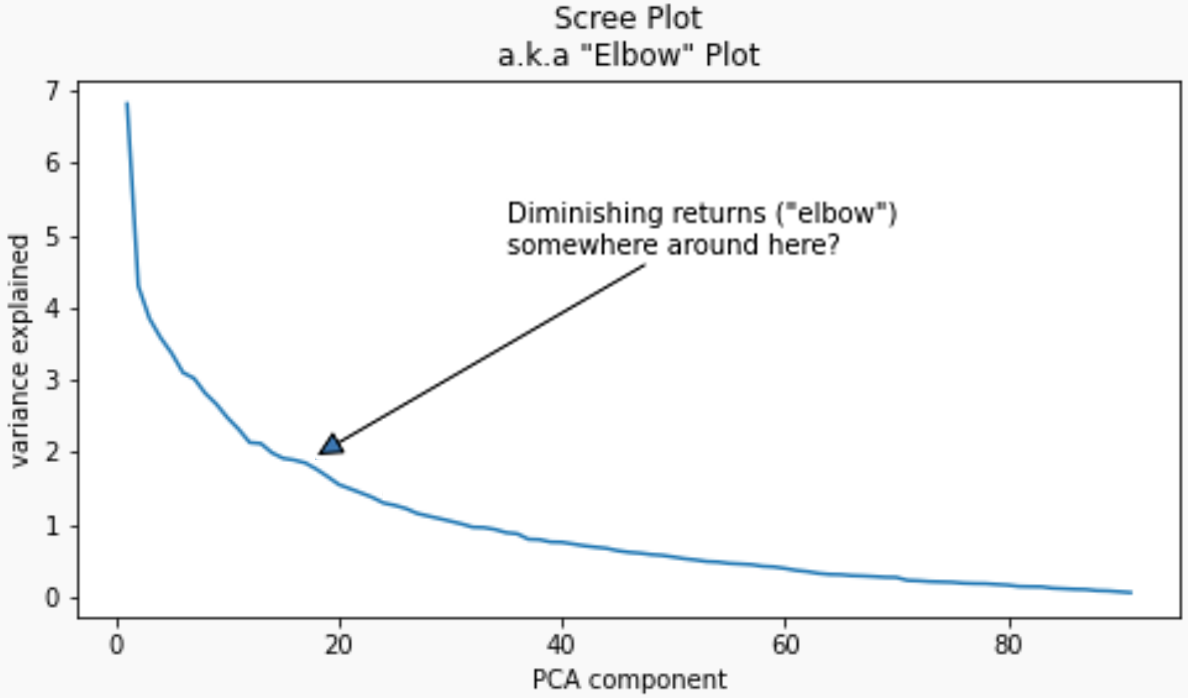
The choice of m is important and can vary from application to application. It depends on various things, like how collinear the predictors are, how truly related they are to the response, etc.

So how should we decide on a value of m?

How many components to keep?

One approach for deciding on the number of components to keep is to just “eyeball” it.

We look for a point of diminishing returns, or “elbow”, in a plot showing the variance explained by each component.



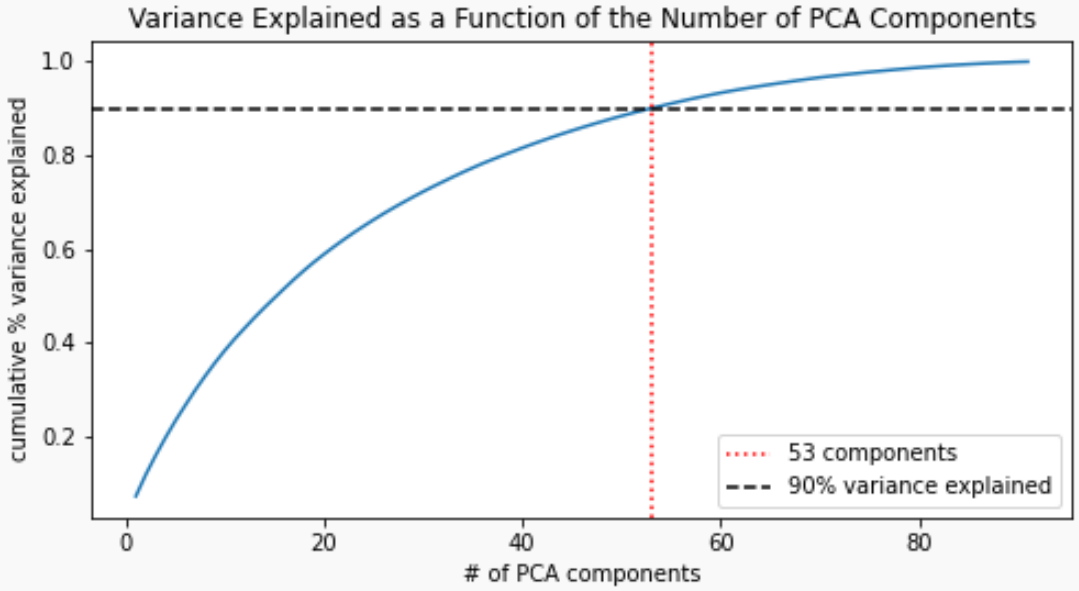


Diminishing returns (“elbow”) somewhere around here?

How many components to keep? (cont.)

Another approach is to specify how much of the total variance we want explained by our *m* components. For example, 90% of the total variance.

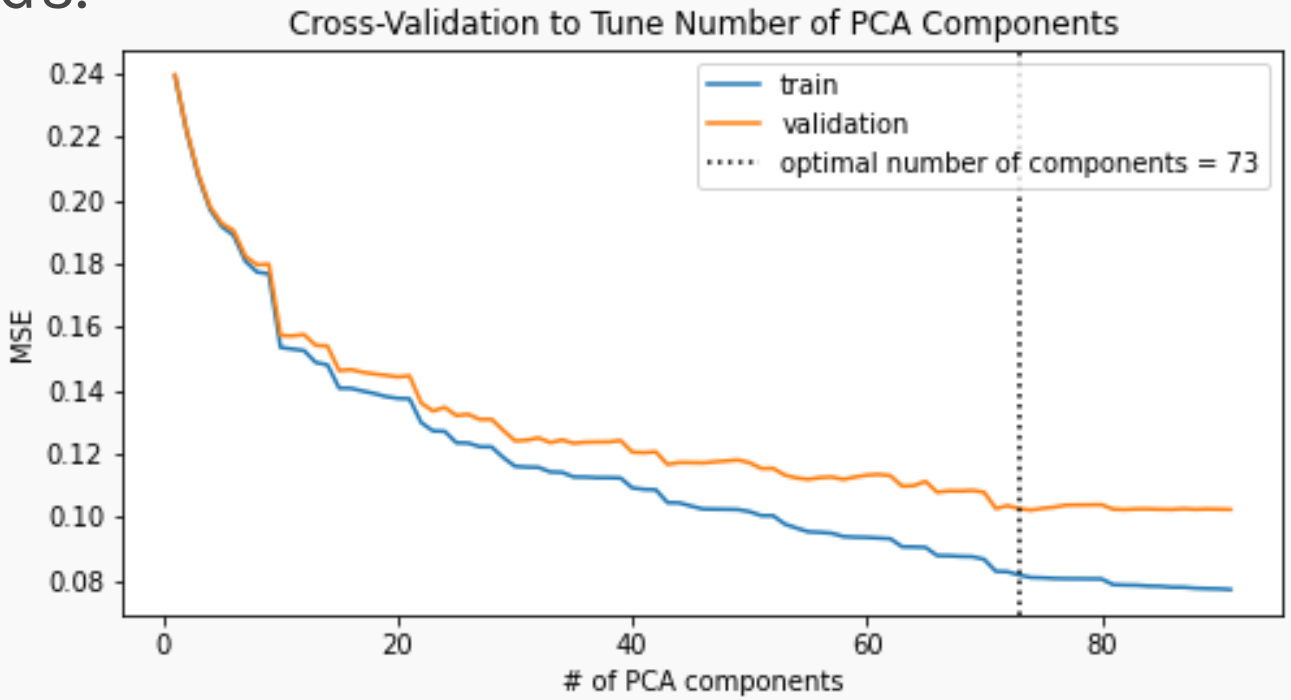
We then inspect the cumulative variance explained by each additional component and stop when we meet out goal.



Cross-validation to tune # of components in PCR

The two previous strategies **did not consider model performance!**

If modeling is your end goal, then you can use cross-validation to *tune* the number of components like any other hyperparameter, selecting the number that received the best mean validation error across all folds.



Interpreting the Results of PCR

A PCR can be interpreted in terms of original predictors…very carefully.

Each estimated β coefficient in the PCR can be *distributed* across the predictors via the associated component vector, *w*.

An example is worth a thousand words:

```
pcaX_df = pd.DataFrame(pcaX, columns=[f'PCA{i}' for i in range(1,X.shape[1]+1)])

PCR_simple = sk.linear_model.LinearRegression().fit(X = pcaX_df[['PCA1']], y = homes['price'])
print("PCA Intercept =",PCR_simple.intercept_.round(2),", PCA Slope =",PCR_simple.coef_[0].round(4))

print("First PCA Component vector (w1):",W[0,:])
```

```
PCA Intercept = 1244.2 , PCA Slope = 0.2049
First PCA Component vector (w1): [ 3.23799939e-01  9.46124307e-01 -7.23671683e-04  9.22493237e-04
  1.00975436e-03]
```

So how can this be transformed back to the original variables?

$$\hat{Y} = \hat{\beta}_0 + \hat{\beta}_1 Z_1 = \hat{\beta}_0 + \hat{\beta}_1(\vec{w}_1^T X) = \hat{\beta}_0 + \left(\hat{\beta}_1 \vec{w}_1^T\right) X$$

$$= 1244.2 + 0.2049 \cdot (0.3238X_1 + 0.9461X_2 - 0.00072\, X_3 + 0.00092X_4 + 0.00101X_5)$$

$$= 1244.2 + 0.0663X_1 + 0.194X_2 - 0.00015\, X_3 + 0.00019X_4 + 0.00021X_5$$

Interpreting the Results of PCR

You can always put the PCR coefficients in terms of the original predictors.

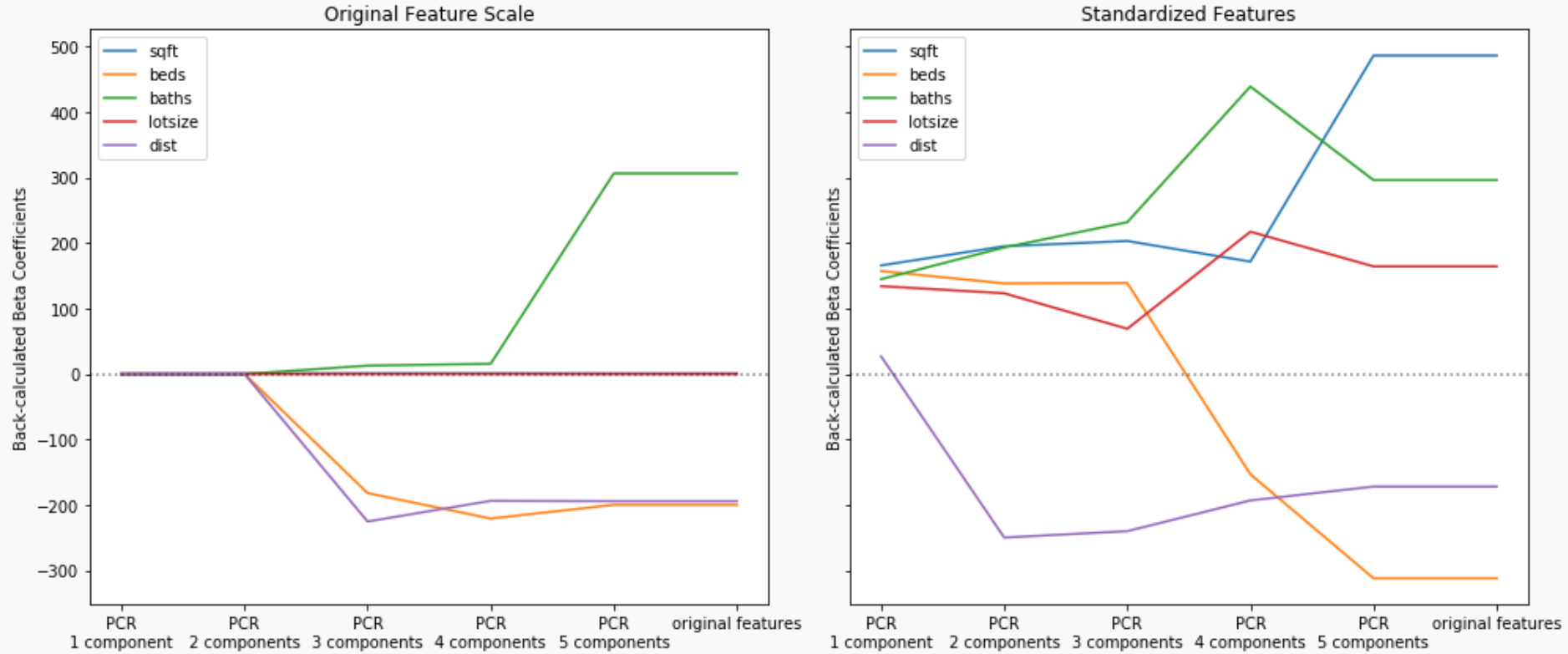
$$\hat{y} = \boldsymbol{Z}\beta_Z = (\boldsymbol{X}\boldsymbol{W}_m)\beta_Z = \boldsymbol{X}(\boldsymbol{W}_m\beta_Z) = \boldsymbol{X}\beta_X$$

$\boldsymbol{X}$ is our original dataset, centered to have a mean of zero
$\boldsymbol{W}_m$ is the matrix whose columns are the first m eigenvectors of $\boldsymbol{X}^\top\boldsymbol{X}$
$\boldsymbol{Z}_m$ is the transformed dataset created by projecting $\boldsymbol{X}$ onto the principal components contained in $\boldsymbol{W}_m$
β_Z are the coefficients for the PCR model
β_X are coefficients in terms of the original predictors

When $m = p$ then the recovered coefficients are identical to those from a model fit on the original predictors.

Coefficient ‘paths’ as more components enter the PCR

Coefficients for the PCR model using all components are identical to coefficients found when fitting on the original predictors.



Notice the effect standardizing has on the coefficients.

A few notes on using PCA

- PCA cons:
    1. PCA *knows nothing about the response variable*. Component vectors as predictors might not be ordered from best to worst!
    2. *Direct* interpretation of coefficients in PCR is lost, so if easy interpretation is important to you, perhaps steer clear.
    3. PCA can often fail to improve the predictive power of a model.
- PCA pros:
    1. Can help avoid the curse of dimensionality and overfitting.
    2. Easy to visualize how predictive your features are of the response variable, especially in the classification setting.
    3. Reduces multicollinearity, and so can improve the computational time when fitting models.